C L I P M I N D A I

Milestone 1 Report

Project Initialization, Design and Core Setup

Video Summarization & Key Moments Detection Platform

Project	**ClipMind AI: AI-powered video transcription, summarization and key moments platform**
Report	Milestone 1 of 4 (Weeks 1 and 2)
Submitted by	Team 1
Domain	Artificial Intelligence
Document type	Project documentation (milestone report)

Foundation: authentication, role-based access, video upload and the FFmpeg processing pipeline.

Contents

1. Milestone at a Glance
2. Timeline
3. Concepts in Short
4. System Design
5. Implementation
6. API Reference
7. Testing and Results
8. Challenges and Resolutions
9. Evaluation Against Milestone Criteria
10. Conclusion and Next Steps
11. Appendix: Screenshots

1. Milestone at a Glance

Milestone 1 built the foundation of ClipMind AI. By the end of these two weeks, Team 1 had a working application in which a user can register, log in securely, see a dashboard that matches their role, upload a video, and watch its processing status change as the backend processes it with FFmpeg.

Table 1: Summary of Milestone 1

Item	Details
Duration	Weeks 1 and 2 (first of four 2-week milestones)
Focus	Project initialization, design process and core setup
Main deliverables	Project structure and Git workflow, architecture and database schema, UI wireframes, React frontend and FastAPI backend setup, registration and login, JWT authentication, role-based access control, video upload with validation and storage, upload history, processing status tracking, FFmpeg processing pipeline, API tests
Technology used	React + Vite + TypeScript, Python FastAPI, PostgreSQL with SQLAlchemy, JWT, FFmpeg, Postman / Swagger, Git and GitHub
Outcome	Working authentication and video management system, ready for the AI features of Milestone 2

1.1 Objectives

- Register an account and log in securely.
- Give each of the four roles (Content Creator, Learner, Educator, Administrator) its own access and dashboard.
- Upload a video, validate it, store it, and keep its record in the database.
- Show upload history and the processing status of every video.
- Start a basic FFmpeg-based video processing pipeline.
- Agree on one project structure, one database approach and one Git workflow for the whole team.

2. Timeline



Figure 1: Eight-week project timeline with Milestone 1 highlighted

Table 2: Week-by-week activities in Milestone 1

Week	Activities	Result
Week 1	Defined project objectives and the media-processing workflow. Agreed the technology stack and the Git rule: no direct changes to main; every member works on a feature branch and opens a Pull Request. Created the shared folder structure. Designed the system architecture and the database schema (User and Video). Prepared UI wireframes (Registration, Login, dashboards, Upload). Set up the frontend and backend environments.	Design approved; both environments running
Week 2	Built registration, login and JWT authentication. Implemented role-based access control for the four roles. Built the React pages: Registration, Login, protected routes, role dashboards, Upload, Upload History, Processing Status. Built the authenticated upload API with validation, storage and ownership checks. Integrated FFmpeg processing with the status flow uploaded, processing, completed or failed. Tested the APIs in Swagger and Postman; added automated tests; reviewed and merged Pull Requests.	Working end-to-end flow from register to processed video

3. Concepts in Short

Table 3: Key concepts used in this milestone

Concept	Plain explanation	Where it is used
REST API	A set of URLs (endpoints) that the frontend calls to ask the backend to do work, using methods like GET and POST.	All communication between React and FastAPI
Password hashing	The password is turned into an irreversible string before saving, so plain-text passwords are never stored.	Registration and login
JWT (JSON Web Token)	A signed token the server gives after a successful login. The frontend sends it with each request as proof of identity.	Every protected API
RBAC	Role-Based Access Control: what a user may do depends on their role.	Dashboards, upload permission, admin pages
ORM (SQLAlchemy)	Lets the backend work with database tables as Python classes instead of raw SQL.	User and Video models
Background task	Work that continues after the API has already replied, so the user is not kept waiting.	FFmpeg video processing
FFmpeg	A command-line tool for converting audio and video.	Re-encoding uploads to H.264 video and AAC audio
Feature branch and Pull Request	Each member works on a separate branch; changes reach main only after review.	Team workflow

4. System Design

4.1 Architecture

ClipMind AI uses a layered design. The React frontend collects input and displays pages. The FastAPI backend holds the business logic, authentication and processing. PostgreSQL stores users and video records. Uploaded files are kept in local storage, and FFmpeg processes them in the background.

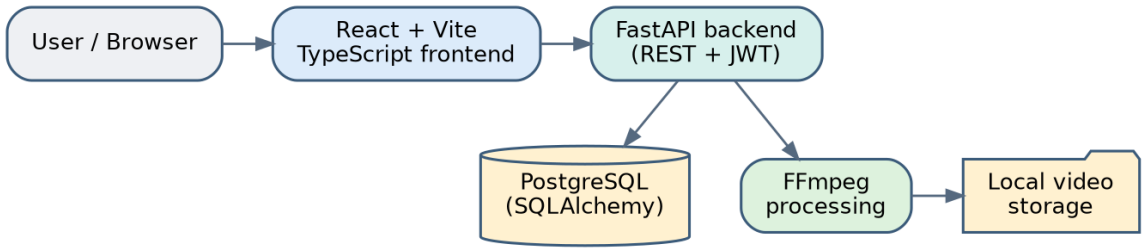


Figure 2: High-level architecture of ClipMind AI (Milestone 1)

4.2 Project structure

```
ClipMind-AI/
├── frontend/    React user interface
├── backend/     FastAPI application, APIs, services, uploads
├── database/    database scripts and notes
├── docs/        diagrams, API notes, documentation
├── tests/       test files
├── .gitignore
└── README.md    setup and run instructions
```

4.3 Database design

Module 1 needs two entities. Each video belongs to exactly one user, so a user can own many videos. The team agreed on PostgreSQL with SQLAlchemy as the single database approach.

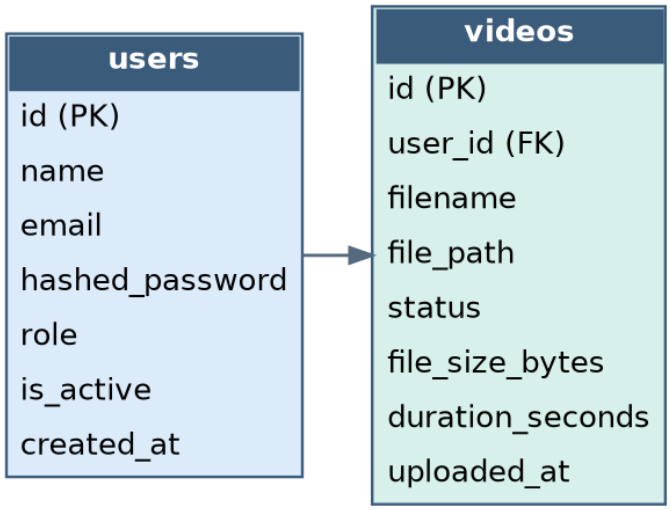


Figure 3: Entity relationship diagram for Milestone 1

Table 4: Database tables created in Milestone 1

Table	Main fields	Purpose
users	id, name, email, hashed_password, role, is_active, created_at	Account details and role; the password is stored only as a hash
videos	id, user_id (FK), filename, file_path, status, file_size_bytes, duration_seconds, uploaded_at	One record per uploaded video, linked to its owner

5. Implementation

5.1 Authentication and JWT

A user registers with name, email, password, confirmed password and role. The backend validates the details, hashes the password and stores the user. At login, the backend checks the password and returns a JWT access token. The frontend keeps the token in the browser (clipmind_access_token) and sends it as Authorization: Bearer <token> on every protected request. The /auth/me endpoint returns the logged-in user, which the frontend uses to restore the session.

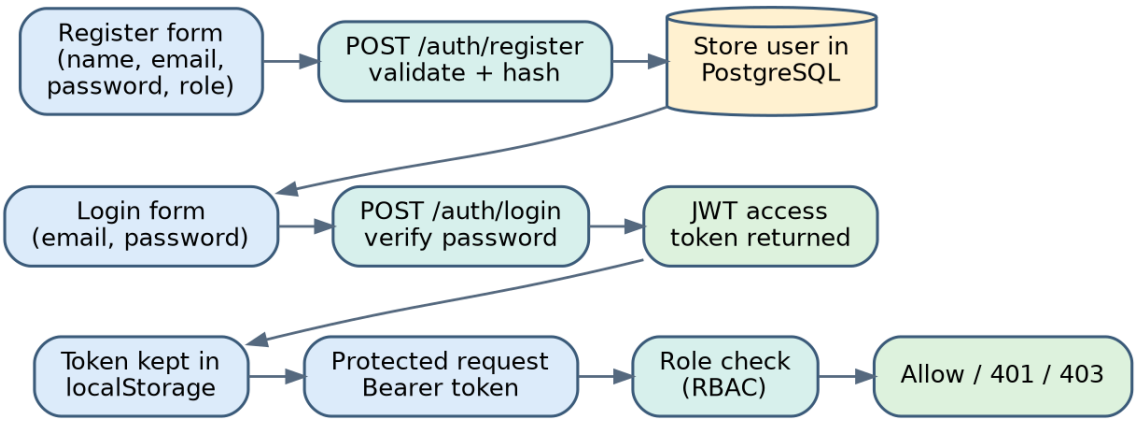


Figure 4: Registration, login and protected-request flow

5.2 Role-based access control

The role is stored with the user account and returned by the backend. The backend uses it to protect APIs, and the frontend uses it to decide which routes and menus to show. A user who opens a page for another role is redirected away.

Table 5: Roles and what each role can access

Role	Navigation available	Upload allowed?
Content Creator	Upload Video, Manage Videos, Processing Status, Transcripts, Upload History	Yes
Educator	Lecture Videos, Transcripts, Summaries, Learning Materials	Yes
Learner	Videos, Transcripts, Summaries, Learning Content	No (view only)
Administrator	Analytics, Users, Roles, Content, AI Processing	No

How access is enforced

The upload endpoint accepts only the Content Creator and Educator roles. Video status and history are filtered by the logged-in user's ID, so one user cannot read another user's video record. Invalid or missing tokens are rejected.

5.3 Video upload, validation and storage

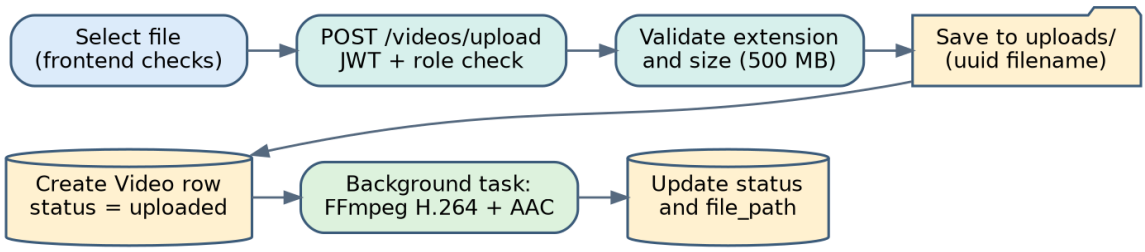


Figure 5: Upload flow from file selection to processing

Table 6: Upload validation rules

Check	Rule	Result when it fails
Authentication	Valid JWT required	401 Unauthorized
Role	Content Creator or Educator only	403 Forbidden
File name	Must be present	400 Bad Request
File type	mp4, mov, avi, mkv or webm	400 Unsupported video format
File size	Maximum 500 MB, read in 1 MB chunks	413 Too large; partial file removed
Storage	Saved under a random UUID file name	500; file removed if database insert fails

5.4 Processing pipeline and status tracking

After the file is saved and its record is created with status uploaded, a background task runs FFmpeg. The task re-encodes the video to H.264 video and AAC audio so that all later steps handle one consistent format. The status in the database tells the frontend where the video is in the process.

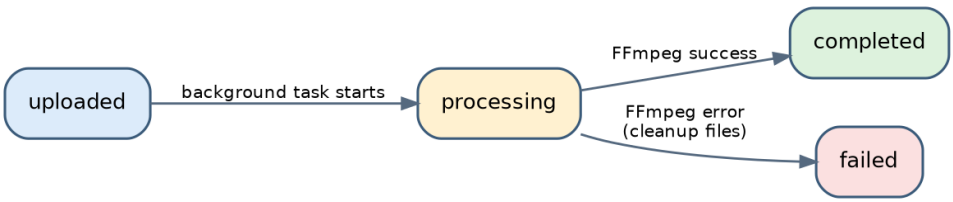


Figure 6: Video status flow

- If FFmpeg fails, the status becomes failed and the incomplete output file is deleted.
- If the source file is missing or the command cannot start, the failure is handled safely without crashing the server.
- Errors shown to users are short and safe; the full FFmpeg output is written only to the server log.

5.5 Frontend pages

- **Register and Login:** forms with validation and clear error messages.
- **Protected routes:** pages open only for logged-in users with the right role.
- **Dashboards:** one per role, opened automatically after login.
- **Upload Video:** file picker with format and size checks before sending.
- **Manage Videos, Upload History and Processing Status:** list the user's videos and their current status.

6. API Reference

Table 7: Endpoints available at the end of Milestone 1

Method	Endpoint	Purpose	Access
POST	/auth/register	Create an account (JSON: name, email, password, confirm_password, role)	Public
POST	/auth/login	Verify credentials and return a JWT	Public
GET	/auth/me	Return the logged-in user	Any logged-in user
POST	/videos/upload	Upload a video (multipart field file)	Creator, Educator
GET	/videos/	List the current user's videos	Logged-in user (own data)
GET	/videos/status	Status of the user's videos	Logged-in user (own data)
GET	/videos/history	Upload history of the user	Logged-in user (own data)
GET	/videos/{video_id}/status	Processing status of one video; 404 if not owned	Owner only
GET	/rbac/...	Role-specific content and admin endpoints used by role pages	Matching role

Swagger was configured so that its Authorize button works with the JWT login, which made endpoint testing easier for the whole team.

7. Testing and Results

Table 8: What was tested in Milestone 1

Area	How it was tested	Result
Authenticated upload	Swagger with a valid token	Passed
Database record creation	Checked the new row in videos after upload	Passed
FFmpeg processing	Background task ran on the uploaded file	Passed
Status flow	uploaded, then processing, then completed (and failed on error)	Passed
Ownership protection	Status endpoint refuses another user's video	Passed
Error handling and cleanup	Failed processing and failed saves remove incomplete files	Passed
Frontend with running backend	Registration, login, protected routes, upload and transcript panel exercised against the live API	Passed
Automated tests	Test suite for upload and processing	

8. Challenges and Resolutions

Table 9: Problems faced and how they were solved

Problem	Cause	Resolution	Status
Upload work blocked	Video upload depended on authentication and database code that was still in a Pull Request	Authentication and database Pull Request was reviewed and merged first; upload and FFmpeg work were then integrated on top	Resolved
Registration returned 422	Frontend sent full_name, backend schema expected name	Payload changed to name; the form label stays "Full name"	Resolved
Swagger could not authenticate	Swagger's OAuth2 settings did not match the login endpoint	Swagger authentication configuration fixed	Resolved
Leftover files after errors	Failed uploads and failed FFmpeg runs left partial files	Cleanup added on every failure path	Resolved
Different work on the same code	Many members changed shared files	Feature branches, Pull Requests and review before merging to main	Followed throughout

9. Evaluation Against Milestone Criteria

Table 10: Milestone 1 evaluation checklist

Criterion	Status
Project initialization and architecture setup completed	Done
Authentication and video upload workflows implemented	Done
Video processing pipeline functional	Done
System design and UI planning completed	Done

10. Conclusion and Next Steps

Milestone 1 delivered a secure, role-aware application with a working upload and FFmpeg pipeline, backed by PostgreSQL. The shared project structure and the Pull Request workflow kept the team's work consistent.

Next (Milestone 2): add the Transcript and Summary data models and migration, integrate Whisper speech-to-text, and build the BART summarization pipeline on top of the stored transcript.

11. Appendix: Screenshots

Add the following screenshots from the running application.

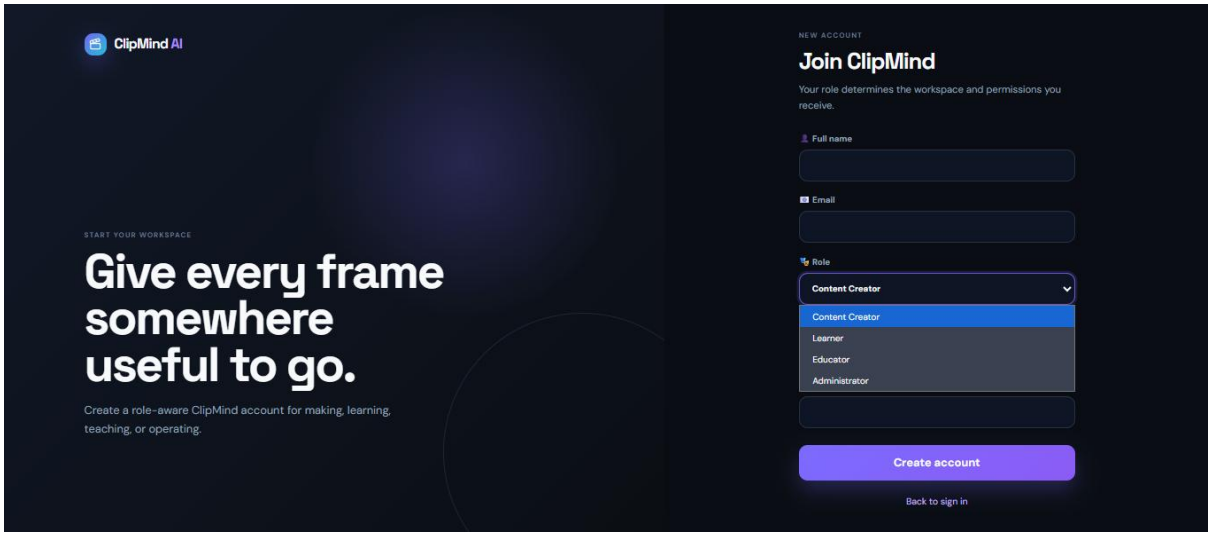
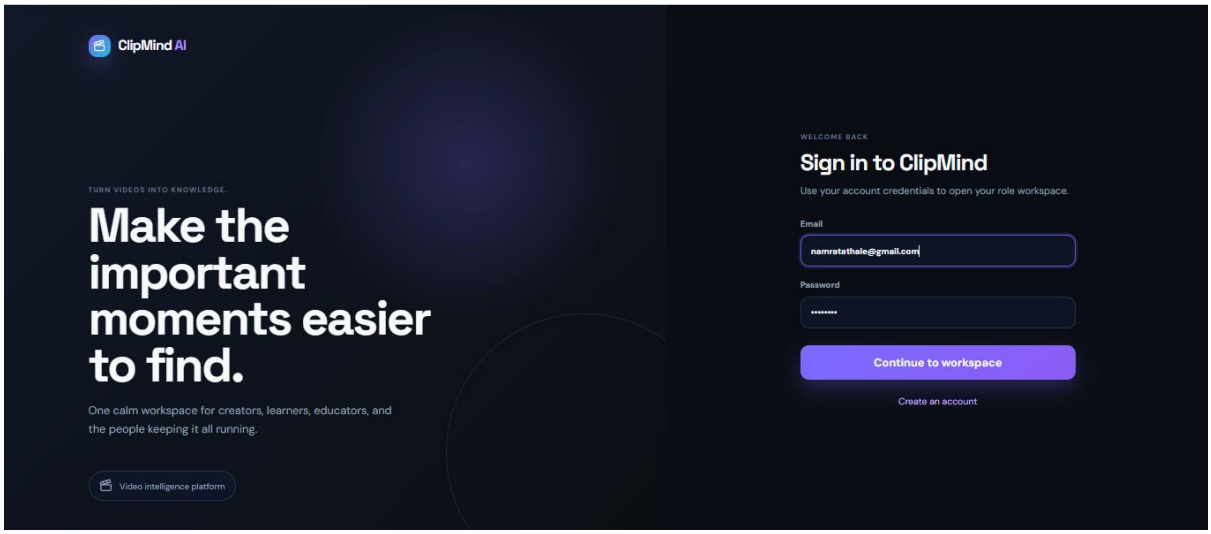


Figure 7: Registration page with role selection



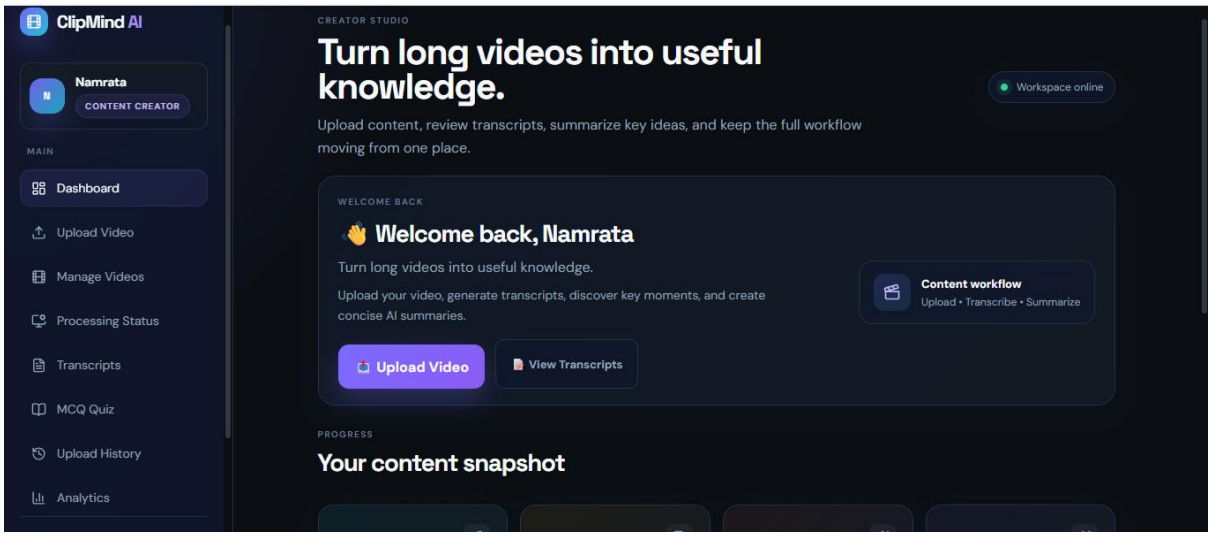


Figure 8: Login page and dashboard

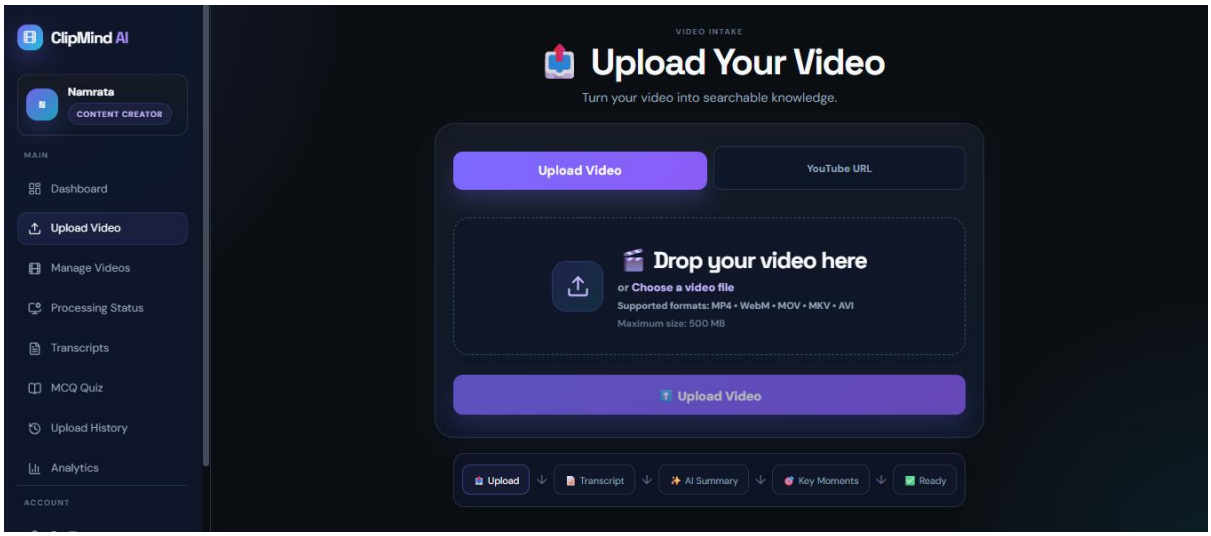


Figure 9: Upload Video page

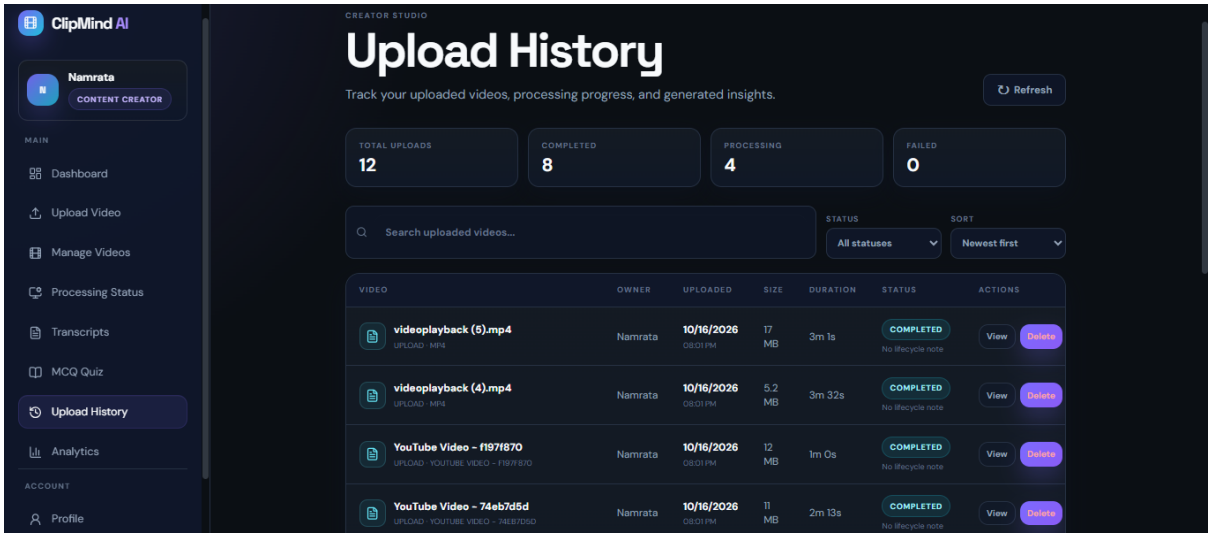


Figure 10: Upload History

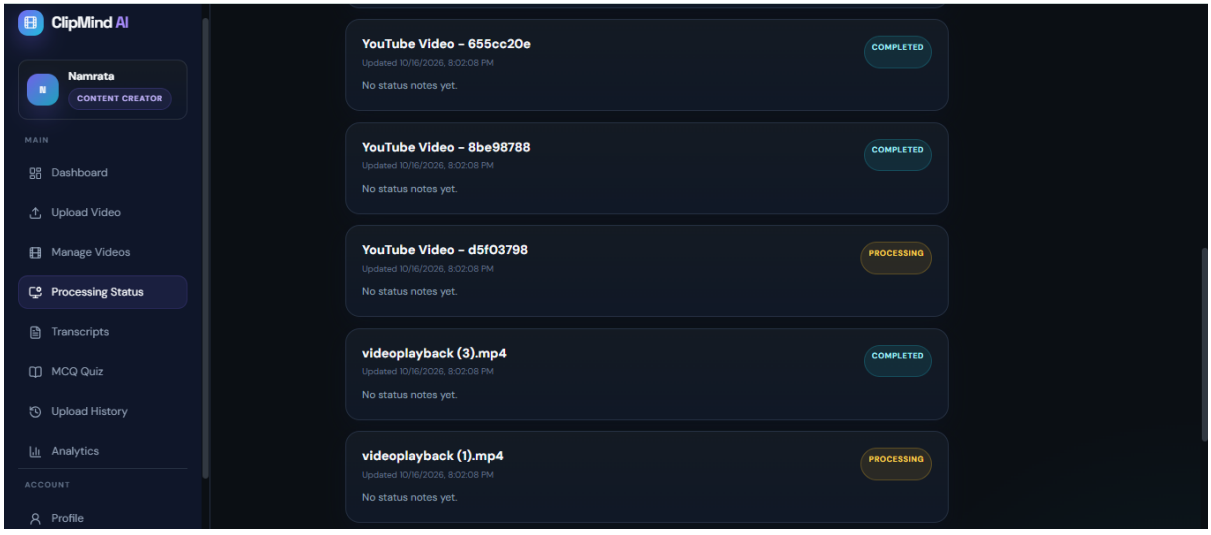


Figure 11: Processing Status

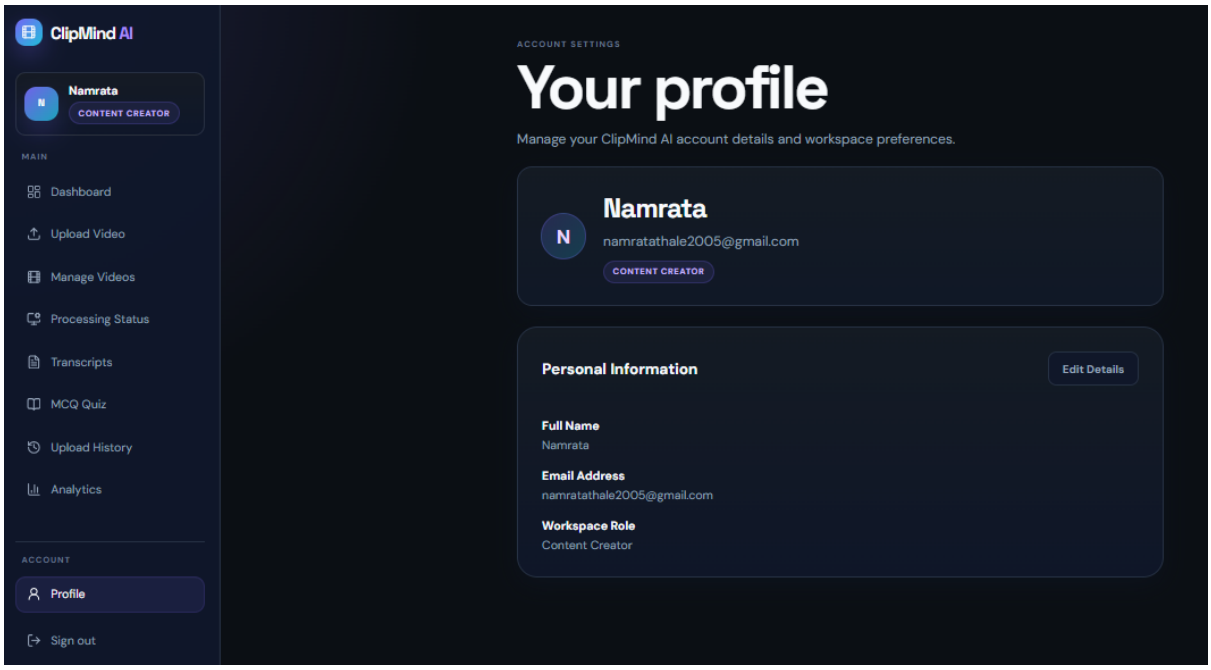


Figure 12: Profile Page